

Software Test Plan (STP)

Personal Wealth Management Software

Project: Personal Wealth Management Software (PWM)

Version: 1.0

Team: Team 9 (Section G)

Authors: Shikshith V (PES1UG24AM400), Sateesh E (PES1UG24AM397), Greeshma K (PES1UG24AM368), Rushikesh Praveen Patil (PES1UG24AM394)

Date: 02-10-2026

Status: Final

This plan is derived from the Software Requirements Specification (SRS) v1.0 and the Software Architecture and Design Specification (SAD) v1.3 of the same project. Requirement IDs and test case IDs are used exactly as they appear in the SRS Requirements Traceability Matrix (Section 8).

Revision History

Version	Date	Author	Change Summary
1.0	02-10-2026	Team 9	Final Software Test Plan based on SRS v1.0 and SAD v1.3.

1. Introduction

Purpose: This document defines the test plan for the Personal Wealth Management Software v1.0. It sets out the objectives, scope, strategy, environment, schedule, resources and responsibilities for testing, and shows how every SRS requirement is verified.

Scope: Testing covers user authentication, asset management (real estate, gold, stocks, cash), valuation and market-data integration, dashboard and reporting, alerts and notifications, administrator capabilities described in SRS Section 2.3, and the non-functional and security requirements of the SRS. Core code of external providers, real money transactions and trading are excluded.

References: SRS v1.0 (Team 9, 07-09-2026) including Section 8 RTM; SAD v1.3 (Team 9); Software Test Plan template; OWASP ASVS and OWASP Top 10; NIST SP 800-63B (password guidance).

Definitions: PWM (Personal Wealth Management), SRS (Software Requirements Specification), SAD (Software Architecture and Design Specification), STP (Software Test Plan), RTM (Requirements Traceability Matrix), UAT (User Acceptance Testing), JWT (JSON Web Token), TLS (Transport Layer Security), p90 (90th percentile), TC (Test Case).

2. Test Items

The following modules, named as in the SRS RTM and SAD Section 3.4, are under test:

- AuthModule: registration, login, logout, password reset, lockout, token handling
- AssetModule: real estate, gold, stock and cash CRUD with validation and ticker lookup
- ValuationService and Market Data Adapters: price and gold-rate refresh, price cache, stale flag, history
- DashboardModule / DashboardUI (React): net worth, allocation, trend charts
- ReportModule: CSV and PDF export
- AlertService and NotificationService: threshold rules, throttling, email and in-app delivery
- AdminModule: health, settings, unlock and disable accounts
- Scheduler Worker: scheduled jobs, locking, catch-up
- Infrastructure: reverse proxy (TLS), Docker deployment, PostgreSQL schema and constraints

3. Features to be Tested

All features below are mapped to SRS requirement IDs. Administrator capabilities described in SRS Section 2.3 are tested through supplementary cases TC-Adm-01 to TC-Adm-03 because the SRS does not assign them numbered requirement IDs.

Req ID	Feature	Priority
PWM-F-001	Register with email and password	High
PWM-F-002	Login and JWT session token	High
PWM-F-003	Password reset by verified email link (30 min, single use)	Medium
PWM-F-010	Add real estate asset	High
PWM-F-011	Add gold asset (form, purity, weight, price)	High
PWM-F-012	Add stock holding (ticker validated)	High
PWM-F-013	Add cash/bank balance	High
PWM-F-014	Edit or delete any asset	High
PWM-F-015	Input validation (types, non-negative values)	High
PWM-F-020	Automatic stock price refresh	High
PWM-F-021	Automatic gold rate refresh	High
PWM-F-022	Manual real estate valuation update	Medium
PWM-F-023	Valuation history by date range	High
PWM-F-024	Cached price and stale flag on feed failure	Medium
PWM-F-030	Net worth total	High
PWM-F-031	Asset allocation chart (sums to 100%)	High
PWM-F-032	Net worth trend chart (1M, 6M, 1Y, All)	Medium
PWM-F-033	CSV and PDF export	Medium
PWM-F-040	Configure alert threshold	Medium
PWM-F-041	Notification on breach within 5 minutes	Medium
PWM-F-042	Alert throttling (once per 24 hours)	Low
PWM-NF-001	Dashboard load p90 <= 2 s	High
PWM-NF-002	Refresh of 500 assets within 5 minutes	Medium
PWM-NF-003	99.5% monthly uptime	High
PWM-NF-004	Usable from 360 px to desktop	Medium
PWM-NF-005	Scale to 1,000 users without redesign	Medium
PRJ-SR-001	TLS 1.2+ for all connections	High
PRJ-SR-002	Salted one-way password hash	High
PRJ-SR-003	JWT expiry and logout invalidation	High
PRJ-SR-004	Per-user authorization (403 on cross-user access)	High
PRJ-SR-005	Account lockout after 5 failed logins for 15 minutes	Medium

4. Features Not to be Tested

- Internal behaviour of third-party stock, gold-rate and email providers (provider APIs are replaced by stubs; only our adapters and retry logic are tested)
- Real money transactions, brokerage or trading execution, tax filing (out of scope in SRS 1.2)

- Multi-user or family sharing and multi-currency support (out of scope in this version)
- Live banking integration (bank and cash balances are entered manually)
- Docker, PostgreSQL, Node.js and browser internals (assumed correct; only our configuration is tested)
- Accuracy of real-estate market values (user-entered estimates; only storage and calculation are tested)

5. Test Approach / Strategy

5.0 Levels and Types

1. Levels:

- 2. Unit tests (Jest): valuation formulas, allocation rounding, alert baseline and throttle logic, validation schemas
- 3. Integration tests (Jest + Supertest on a PostgreSQL service container): API with database, market adapters against stubbed providers, scheduler jobs, notification outbox
- 4. System tests (end-to-end, Selenium/Playwright): complete user journeys in the browser
- 5. Acceptance tests (UAT): scenarios from the SRS acceptance criteria run by a team member who did not develop the feature

1. Types:

- 2. Functional testing of every SRS requirement using the acceptance criteria AC-PWM-F-xxx
- 3. Regression testing: automated suite run in GitHub Actions on every pull request, and a full run before each release
- 4. Performance testing (JMeter): dashboard p90, 500-asset refresh, 1,000-user load
- 5. Usability and responsive testing: 360 px to desktop, four browsers
- 6. Reliability testing: provider failure, worker restart, email failure, container restart

5.0.1 Entry and Exit Criteria

Entry criteria: Stable build deployed in the test environment, database migrations applied, test data and provider stubs available, smoke test passed.

Exit criteria: 100% of planned test cases executed; every High-priority functional requirement implemented and verified; 0 critical or high defects open; no critical NFR failure; the RTM shows all planned test cases passing (as required by SRS Section 6).

5.1 Security Validation

- TLS: plain HTTP refused, only TLS 1.2+ accepted, certificate validation on provider calls (TC-Sec-01)
- Password storage is an argon2id salted hash, nothing in plaintext in the DB or logs (TC-Sec-02)
- JWT expiry, logout revocation, invalidation after password reset (TC-Sec-03)
- Authorization: cross-user access returns 403 for assets, valuations, alerts and reports (TC-Sec-04)
- Lockout after 5 failed logins for 15 minutes, with audit logging (TC-Sec-05)
- Cookie flags, Origin check, security headers (TC-Sec-06)
- Input fuzzing and injection tests on login, asset forms and query parameters; CSV formula injection check (TC-Sec-07, TC-Export-01)
- Administrator isolation: admins cannot read user portfolio data (TC-Adm-02)
- Dependency and image scan in CI (npm audit and a container scan) reviewed before release

6. Test Environment

Hardware: Developer laptops and one Linux host or cloud VM (2 vCPU, 4 GB RAM minimum) for the staging environment; test devices or browser emulation for 360 px mobile screens.

Software: PWM application v1.0 (React frontend, Node.js 18+ and Express API, scheduler worker), PostgreSQL 14+, Docker Compose, nginx reverse proxy with a test TLS certificate. Browsers: Chrome, Firefox, Safari, Edge (latest versions).

Stubs: Mock stock and gold price servers that can return normal data, delayed data, errors and timeouts; an SMTP test server (for example MailHog) in place of a real email provider.

Tools: Jest and Supertest (unit and API), Selenium or Playwright (UI automation), Postman (manual API checks), JMeter (performance), GitHub Actions (CI), Jira or GitHub Issues (defect tracking), OWASP ZAP (security scan).

Test Data: Three seeded users (User A, User B for isolation tests, and one administrator). Sample portfolio per user: RELIANCE 10 shares at 2,800.00 (NSE), 20 g of 22K gold coin, flat valued 6,500,000.00, and a cash account. Valuation history and snapshots seeded over 12 months for trend tests. A generated set of 1,000 users with about 20 assets each for scale tests and 500 market holdings for the refresh test. All data is dummy; no real financial or personal data is used.

7. Test Schedule

Dates follow the SAD implementation order and can be adjusted by the team lead once the build schedule is confirmed.

Milestone	Date
Test case design (from SRS acceptance criteria and SAD)	05-Oct-2026
Environment and stub setup	07-Oct-2026
Cycle 1: Authentication and Asset Management (unit, API)	08-Oct-2026 to 11-Oct-2026
Cycle 2: Valuation, Dashboard, Reports, Alerts, Scheduler (integration, system)	12-Oct-2026 to 16-Oct-2026
Cycle 3: Performance, Security, Usability, Admin	17-Oct-2026 to 20-Oct-2026
Regression and defect retest	20-Oct-2026
UAT	22-Oct-2026 to 25-Oct-2026
Test Summary Report	26-Oct-2026

8. Test Deliverables

- Test Plan (this document)
- Test Cases (Appendix A; manual and automated)
- Test Scripts (Jest, Supertest, Selenium/Playwright, JMeter)
- Test Data and stub configurations
- Test Execution Logs and CI reports
- Defect Reports
- Requirements Traceability Matrix with test status
- Test Summary Report

9. Roles and Responsibilities

Roles follow the team role mapping in the SRS. Every member also executes tests on modules they did not write.

Role	Name	Responsibility
QA Lead / Security Testing	Rushikesh Praveen Patil	Prepare and maintain the plan, coordinate execution, security and CI test pipeline, performance tests, Test Summary Report
Test Engineer (API and Logic)	Sateesh E	Design and execute authentication, asset, valuation and alert test cases; fix and triage backend defects

Role	Name	Responsibility
Test Engineer (Data)	Greeshma K	Test data and database tests: schema constraints, valuation history, snapshots, scale data; support defect fixes
Test Engineer (UI) / Product Owner	Shikshith V	UI, responsive and usability tests, dashboard tests; approve test results and sign off readiness

10. Risks and Mitigation

Risk	Mitigation
Delay in stable build delivery	Request early smoke builds; test finished modules first following the SAD checklist order
Test environment downtime	Keep Docker Compose setup scripted so the environment can be rebuilt on any laptop or cloud VM
Real provider APIs are unavailable, rate limited or change format	Use stubs for all automated tests; run only a small manual check against the real provider
Time-dependent behaviour is slow to test (24-hour throttle, 30-minute reset link, 24-hour token, trading hours)	Make clocks configurable or injectable and use short test settings; verify the real defaults with a configuration review
Performance results differ from production	Run performance tests on a production-like Docker host and record the hardware used
Cross-user data leakage missed	Run every API test against two users; add automated authorization tests for each endpoint
Small team and many test cases	Automate API and unit tests in CI; reserve manual effort for UI, usability and UAT

11. Assumptions & Dependencies

- The build follows SAD v1.3, including the endpoints in SAD 4.3 and the data model in SAD 4.4
- Provider stubs and the SMTP test server are available before execution starts
- Test data (users, assets, history) is generated by scripts before each cycle
- The final choice of stock, gold and email providers (SAD open issues) does not change the adapter interfaces
- SRS acceptance criteria and test case IDs remain unchanged during testing
- Administrator capabilities are traced to SRS Section 2.3 unless the SRS is updated with numbered IDs.

12. Suspension & Resumption Criteria

1. Suspend testing if:

- 2. The test environment is unavailable for more than 4 hours
- 3. The build is too unstable and blocks more than 30% of test cases
- 4. A critical defect (data loss, cross-user data exposure, login failure for all users) is found; testing of the affected module stops until it is fixed

1. Resume testing when:

- 2. Blocking defects are resolved and a new build passes the smoke test
- 3. The environment is stable again and test data is reloaded

13. Test Case Management & Traceability

The RTM in SRS Section 8 maps each requirement to its test case. This plan keeps the same IDs and adds supplementary design-derived cases from the SAD. Status values: Planned, In Progress, Passed, Failed, Blocked. All start as Planned (N in the SRS RTM).

Req ID	Requirement (short)	Test Case(s)	Status
PWM-F-001	Register with email and password	TC-Auth-01	Planned
PWM-F-002	Login and JWT session token	TC-Auth-02	Planned
PWM-F-003	Password reset by verified email link (30 min, single use)	TC-Auth-03	Planned
PWM-F-010	Add real estate asset	TC-RE-01	Planned
PWM-F-011	Add gold asset (form, purity, weight, price)	TC-Gold-01	Planned
PWM-F-012	Add stock holding (ticker validated)	TC-Stock-01	Planned
PWM-F-013	Add cash/bank balance	TC-Cash-01	Planned
PWM-F-014	Edit or delete any asset	TC-Asset-01	Planned
PWM-F-015	Input validation (types, non-negative values)	TC-Asset-02	Planned
PWM-F-020	Automatic stock price refresh	TC-Val-01	Planned
PWM-F-021	Automatic gold rate refresh	TC-Val-02	Planned
PWM-F-022	Manual real estate valuation update	TC-Val-03	Planned
PWM-F-023	Valuation history by date range	TC-Val-04	Planned
PWM-F-024	Cached price and stale flag on feed failure	TC-Val-05	Planned
PWM-F-030	Net worth total	TC-Dash-01	Planned
PWM-F-031	Asset allocation chart (sums to 100%)	TC-Dash-02	Planned
PWM-F-032	Net worth trend chart (1M, 6M, 1Y, All)	TC-Dash-03	Planned
PWM-F-033	CSV and PDF export	TC-Export-01	Planned
PWM-F-040	Configure alert threshold	TC-Alert-01	Planned
PWM-F-041	Notification on breach within 5 minutes	TC-Alert-02	Planned
PWM-F-042	Alert throttling (once per 24 hours)	TC-Alert-03	Planned
PWM-NF-001	Dashboard load p90 <= 2 s	TC-Perf-01	Planned
PWM-NF-002	Refresh of 500 assets within 5 minutes	TC-Perf-02	Planned
PWM-NF-003	99.5% monthly uptime	Ops-Uptime	Planned
PWM-NF-004	Usable from 360 px to desktop	TC-UX-01	Planned
PWM-NF-005	Scale to 1,000 users without redesign	TC-Scale-01	Planned
PRJ-SR-001	TLS 1.2+ for all connections	TC-Sec-01	Planned
PRJ-SR-002	Salted one-way password hash	TC-Sec-02	Planned
PRJ-SR-003	JWT expiry and logout invalidation	TC-Sec-03	Planned
PRJ-SR-004	Per-user authorization (403 on cross-user access)	TC-Sec-04	Planned
PRJ-SR-005	Account lockout after 5 failed logins for 15 minutes	TC-Sec-05	Planned
SRS 2.3 / SAD 4.3	Administrator capabilities (no SRS requirement ID)	TC-Adm-01, TC-Adm-03	Planned
SRS 2.3 / SAD 3.9	Administrator capabilities (no SRS requirement ID)	TC-Adm-02	Planned

Req ID	Requirement (short)	Test Case(s)	Status
SAD 3.9	Design-derived test (SAD)	TC-Sec-06, TC-Sec-07	Planned
SAD 4.6	Design-derived test (SAD)	TC-Job-01	Planned
SAD 4.6 / 4.7	Design-derived test (SAD)	TC-Notif-01	Planned

Test case details (steps, expected results, level) are in Appendix A.

14. Test Metrics & Reporting

1. Metrics collected:

- 2. % of test cases executed
- 3. % passed / failed / blocked
- 4. Defect density (defects per module) and defect aging
- 5. Requirement coverage (requirements with at least one passed test case)
- 6. Automated test coverage reported by Jest, and CI pass rate
- 7. Performance results: p90 dashboard load, refresh job duration, uptime

1. Reports:

- 2. Daily execution status during each cycle
- 3. Final Test Summary Report at the end of UAT

15. Approvals

Role	Name	Signature / Date
QA Lead	Rushikesh Praveen Patil	
Dev Lead (Backend)	Sateesh E	
Product Owner / Team Lead	Shikshith V	
Course Coordinator		

Appendix A: Test Case Catalogue

Cases TC-Auth to TC-Sec-05, TC-Perf, TC-UX, TC-Scale and Ops-Uptime use the IDs defined in the SRS RTM. TC-Sec-06, TC-Sec-07, TC-Adm-01 to 03, TC-Job-01 and TC-Notif-01 are added from the SAD.

Test Case ID	Requirement	Scenario / Steps	Expected Result	Level
TC-Auth-01	PWM-F-001	Register with a valid email and strong password. Repeat with a weak password (under 10 chars, common password) and with an already-registered email.	Valid input creates the account (201). Weak password rejected (400, field message). Duplicate email rejected (409).	API / System
TC-Auth-02	PWM-F-002	Log in with valid credentials, then with a wrong password and an unknown email.	Valid login sets the httpOnly auth cookie (JWT). Invalid attempts return 401 with identical messages for wrong password and unknown email.	API / System
TC-Auth-03	PWM-F-003	Request a password reset; use the emailed link within 30 minutes; reuse the same link; try a link older than 30 minutes.	First use within 30 minutes succeeds. Reused or expired link is rejected. Old sessions stop working after reset.	API / System
TC-RE-01	PWM-F-010	Add a real estate asset (name, location, purchase value, purchase date, current estimated value).	Asset saved and shown in the real estate list with the entered values; net worth includes it.	API / System
TC-Gold-01	PWM-F-011	Add gold assets for each form (jewellery, coin, bar) with purity and weight. Try purity 0 and 25, and a negative weight.	Valid gold saved. Purity outside 1-24 and negative weight rejected with a clear error.	API / System
TC-Stock-01	PWM-F-012	Add a holding with a known ticker (e.g. RELIANCE, NSE). Add one with an unknown ticker.	Known ticker saved. Unknown ticker rejected (422). Ticker lookup (GET /api/market/symbols) returns matches.	API / System
TC-Cash-01	PWM-F-013	Add a cash/bank balance with an account label and balance.	Balance saved and reflected in the dashboard total.	API / System
TC-Asset-01	PWM-F-014	Edit the value of an asset, then delete another asset, in each asset class.	Dashboard total and today's snapshot change immediately. Deleted assets are excluded from totals and exports.	API / System
TC-Asset-02	PWM-F-015	Submit wrong data types, empty required fields, negative quantity/weight/price.	All rejected (400) with field-level messages. Database CHECK constraints also block negative values when the API is bypassed.	Unit / API
TC-Val-01	PWM-F-020	Hold 3 stocks. Run the stock-refresh job against the stubbed price API with changed prices.	price_cache and each holding's value update to quantity x new price; valuation rows written with source MARKET.	Integration
TC-Val-02	PWM-F-021	Run the gold-refresh job with a stub that returns the 24K rate only; hold 22K gold.	22K value = weight x (rate24 x 22 / 24). Valuation row written.	Integration
TC-Val-03	PWM-F-022	Call PUT /api/assets/:id/valuation for a real estate asset.	New value stored with timestamp; MANUAL valuation row written; net worth updated.	API
TC-Val-04	PWM-F-023	Create several valuations over different dates. Query /api/valuations/history with from/to.	Only rows inside the date range are returned, in order, for the owning user.	API / DB

Test Case ID	Requirement	Scenario / Steps	Expected Result	Level
TC-Val-05	PWM-F-024	Make the stub provider fail (timeout / 500) after a successful refresh.	3 retries with backoff, cached price stays in use, API returns isStale=true with priceAsOf, UI shows the stale badge. Next success clears it.	Integration / System
TC-Dash-01	PWM-F-030	Add assets across all four classes; open the dashboard.	Displayed net worth equals the sum of current values of all non-deleted assets.	System
TC-Dash-02	PWM-F-031	Use values that cause rounding (e.g. 3 classes at 33.33%).	Allocation percentages sum to exactly 100.00 (remainder added to the largest slice); chart renders.	Unit / System
TC-Dash-03	PWM-F-032	Select 1M, 6M, 1Y and All on the trend chart using seeded snapshots.	Chart redraws for each range with the correct data points.	System
TC-Export-01	PWM-F-033	Export CSV and PDF with seeded assets and valuation history.	Both files contain every current asset record and the valuation history. CSV cells starting with = + - @ are neutralized.	API / System
TC-Alert-01	PWM-F-040	Create a net-worth alert at 5% and an asset alert at 10%. Run the next valuation cycle with a 6% net-worth move.	Thresholds saved and applied on the next cycle. Change measured against the baseline 24 hours earlier; net-worth alert fires, asset alert does not.	API / Integration
TC-Alert-02	PWM-F-041	Trigger a threshold breach and time the delivery of the email and the in-app notification.	In-app notification created at once; email delivered within 5 minutes of breach detection. Failed email retried up to 5 times.	Integration / System
TC-Alert-03	PWM-F-042	Trigger the same alert twice within 24 hours, then again after 24 hours.	Second trigger inside 24 hours creates no event or notification. Trigger after 24 hours fires.	Integration
TC-Perf-01	PWM-NF-001	JMeter: 50 concurrent users on GET /api/dashboard and page load in a production-like environment.	90th percentile page load <= 2 s.	Performance
TC-Perf-02	PWM-NF-002	Seed 500 market-linked holdings; run stock and gold refresh jobs against the stub.	Each job completes within 5 minutes; duration recorded in job_runs.	Performance
Ops-Uptime	PWM-NF-003	Review uptime monitor reports for the staging period and test container restart and health checks.	Uptime >= 99.5% for the month; failed container restarts automatically.	Operational
TC-UX-01	PWM-NF-004	Open all main screens at 360, 768 and 1280 px in Chrome, Firefox, Safari and Edge.	No horizontal page scroll; tables and charts scroll in their own container; forms usable.	Usability
TC-Scale-01	PWM-NF-005	Load test with 1,000 simulated users with about 20 assets each.	Response times remain within NF-001 limits; no schema change needed.	Performance
TC-Sec-01	PRJ-SR-001	Request every endpoint over plain HTTP; scan TLS versions on the proxy; check provider adapters.	HTTP refused. Only TLS 1.2+ accepted; adapters use HTTPS with certificate validation.	Security
TC-Sec-02	PRJ-SR-002	Register a user and inspect users.password_hash.	Value is an argon2id hash with a per-password salt; no plaintext anywhere (DB, logs).	Security
TC-Sec-03	PRJ-SR-003	Use an expired token, a token after logout, and a token issued before a password reset.	All rejected with 401. Default expiry is 24 hours.	Security / API

Test Case ID	Requirement	Scenario / Steps	Expected Result	Level
TC-Sec-04	PRJ-SR-004	As User A, request User B's asset, valuation, alert and report by ID.	403 returned for every cross-user request; attempt is audited.	Security / API
TC-Sec-05	PRJ-SR-005	Make 5 consecutive wrong logins, then a 6th with the correct password; wait 15 minutes.	6th attempt blocked (423) with retry time and logged in audit_log. Login works after 15 minutes.	Security / API
TC-Sec-06	SAD 3.9	Send a state-changing request with a foreign Origin; inspect cookie flags and response headers.	Request blocked; cookie is httpOnly, Secure, SameSite=Strict; CSP and HSTS headers present.	Security
TC-Sec-07	SAD 3.9	Inject SQL and script payloads into text fields, query parameters and asset names; fuzz the login and asset forms.	No SQL execution or script rendering; invalid input rejected; no stack traces returned.	Security
TC-Adm-01	SRS 2.3 / SAD 4.3	Call /api/admin/* as a normal user and as an administrator (health, settings, users).	Normal user gets 403. Administrator can read health and update settings.	API
TC-Adm-02	SRS 2.3 / SAD 3.9	As administrator, try to read any user's assets, valuations or reports.	Access denied; the admin user list contains no portfolio data.	Security / API
TC-Adm-03	SRS 2.3 / SAD 4.3	Lock a test account, unlock it, then disable it as administrator.	Account unlocked and disabled correctly; disabled user cannot log in; each action appears in audit_log.	API
TC-Job-01	SAD 4.6	Start two worker instances and trigger the same job; restart the worker mid-job.	Only one instance runs the job (advisory lock); a missed job within 6 hours runs once on restart.	Integration
TC-Notif-01	SAD 4.6 / 4.7	Make the email stub fail, then recover it.	Notification retried at 1, 2, 4, 8 minutes; status FAILED after the last attempt; in-app copy still exists.	Integration